

Diseño de prompts

Cómo escribir, parametrizar y mejorar los prompts que hacen que un modelo generativo resuma los tickets de Nube Sur de forma útil y predecible.

APRENDIZAJE ESPERADO 3 · TEXTUAL DEL PLAN FORMATIVO

FORMULAR PROMPTS EFECTIVOS PARA MODELOS GENERATIVOS EN ESCENARIOS ZERO-SHOT O FEW-SHOT, CONSIDERANDO LA TAREA SOLICITADA Y LOS PARÁMETROS DE GENERACIÓN DISPONIBLES.

PLAN FORMATIVO

PF1822 · Especialización en Desarrollo con IA

MÓDULO

MA04576 · INTRODUCCIÓN A MODELOS GENERATIVOS Y CONSUMO POR API · 21 h

CASO DE ESTUDIO

Nube Sur, empresa ficticia

TIEMPO DE LECTURA

20 minutos, más la autocomprobación

Qué vas a lograr

Al terminar esta lectura tendrás la base para demostrar los criterios de evaluación del aprendizaje esperado 3, textuales del plan formativo:

- 3.1

EXPLICA LA FUNCIÓN DE UN PROMPT EN LA INTERACCIÓN CON MODELOS GENERATIVOS, DISTINGUIENDO ENTRE ESTRATEGIAS ZERO-SHOT Y FEW-SHOT, E IDENTIFICANDO SUS VENTAJAS Y LIMITACIONES BÁSICAS.
- 3.2

ELABORA PROMPTS ADECUADOS PARA TAREAS ESPECÍFICAS, COMO RESUMEN, GENERACIÓN DE CÓDIGO O REFORMULACIÓN DE TEXTO, SELECCIONANDO LA ESTRATEGIA DE PROMPTING MÁS APROPIADA SEGÚN EL OBJETIVO.
- 3.3

AJUSTA UN PROMPT DE FORMA ITERATIVA CON BASE EN LOS OUTPUTS OBTENIDOS, DESCRIBIENDO LOS CAMBIOS REALIZADOS EN TÉRMINOS DE CLARIDAD, ESTRUCTURA O CONTROL DEL COMPORTAMIENTO DEL MODELO.

Cómo está organizada

1	Qué es un prompt	6	Parámetros
2	Tipos de prompting	7	Limitaciones
3	Buenas prácticas	8	Iterar
4	Formato de salida	+ En síntesis, práctica, autocomprobación y glosario	
5	Ejemplos prácticos		

Contenidos del plan y dónde se tratan

Contenido del plan formativo (textual)	Dónde
3. DISEÑO DE PROMPTS PARA APIS GENERATIVAS	Antes de empezar
QUÉ ES UN PROMPT.	Sección 1
ESTRUCTURA BÁSICA DE ENTRADA PARA MODELOS DE LENGUAJE.	Sección 1
TIPOS DE PROMPTING: ZERO-SHOT, FEW-SHOT E INSTRUCTIONAL PROMPTING.	Sección 2
PRÁCTICAS RECOMENDADAS: CLARIDAD, DELIMITACIÓN DEL ROL, CONTEXTO, RESTRICCIONES Y EVITAR AMBIGÜEDAD.	Sección 3
FORMATEO DE SALIDAS: LISTAS, JSON Y ESTRUCTURA TABULAR.	Sección 4
EJEMPLOS PRÁCTICOS: PROMPT PARA GENERAR CÓDIGO, RESUMEN, VALIDACIÓN Y REFORMULACIÓN.	Sección 5
PROMPT Y PARÁMETROS: CÓMO INFLUYEN TEMPERATURE, MAX_TOKENS Y TOP_P.	Sección 6
LIMITACIONES: ALUCINACIONES, FALTA DE CONTROL Y SENSIBILIDAD AL WORDING.	Sección 7
ITERACIÓN Y AJUSTE DE PROMPTS SEGÚN RESULTADOS OBTENIDOS.	Sección 8

Antes de empezar

CONTENIDO DEL PLAN 3. DISEÑO DE PROMPTS PARA APIS GENERATIVAS

En el aprendizaje anterior construiste `ClienteIA`, una clase que habla con un modelo por API. Ahora la pregunta es qué le dices. Un modelo de lenguaje hace lo que el texto de entrada le permite entender: si la instrucción es vaga, la respuesta también lo será. Diseñar un prompt es escribir una **especificación**, igual que cuando describes una función antes de programarla: qué entra, qué debe salir y qué no puede pasar.

Esta lectura sigue el caso de **Nube Sur**, una empresa ficticia de software cuya mesa de ayuda recibe decenas de tickets al día. El equipo quiere un resumidor que entregue una oración clara por ticket, para que el jefe de soporte priorice sin leer cada mensaje completo. El primer intento fue un prompt de tres palabras y el resultado fue inservible. Al terminar esta lectura vas a poder explicar por qué y cómo corregirlo.

CÓMO SE CONSTRUYE UN BUEN PROMPT



Los ejemplos usan la estructura de mensajes de la API de chat que ya conoces. Puedes probarlos con tu `ClienteIA` o en el notebook guiado del curso.

1 Qué es un prompt

CONTENIDO DEL PLAN QUÉ ES UN PROMPT. · ESTRUCTURA BÁSICA DE ENTRADA PARA MODELOS DE LENGUAJE.

Un **prompt** es la entrada que recibe un modelo generativo para producir su respuesta. En un modelo de lenguaje es texto, y cumple dos funciones a la vez: le dice al modelo **qué tarea** hacer y le entrega **los datos** sobre los que debe hacerla. El modelo no ejecuta el prompt como un programa; genera, token a token, la continuación más probable dada esa entrada. Por eso cada palabra influye.

Las APIs de chat ordenan la entrada en una lista de **mensajes**, cada uno con un rol. El mensaje de **sistema** (`system`) fija el comportamiento general: quién es el asistente, qué reglas sigue y en qué formato responde. El mensaje de **usuario** (`user`) trae la tarea concreta y los datos. Los mensajes del **asistente** (`assistant`) son respuestas anteriores del modelo, y también sirven para mostrarle ejemplos, como verás en la sección 2.

```
mensajes = [
    {"role": "system", "content": "Eres analista de soporte de Nube Sur. "
                                     "Resume cada ticket en una oración."},
    {"role": "user",
     "content": "Ticket: No puedo descargar la factura de agosto "
                 "desde el portal; el botón queda cargando."},
]
respuesta = cliente.generar(mensajes, temperature=0.2, max_tokens=60)
```

EJEMPLO · NUBE SUR · Separar reglas y datos

El prompt que rechazó el jefe de soporte, Resume esto: <ticket>, mezclaba todo en una línea y no fijaba ninguna regla. Al pasar a dos mensajes, las reglas quedan en el de sistema, iguales para todos los tickets, y el mensaje de usuario solo cambia el ticket. Así es más fácil comparar resultados y mantener el prompt.

IDEA CLAVE

El modelo no sabe nada de Nube Sur, de sus productos ni de lo que el jefe de soporte considera un buen resumen. Todo lo que necesita saber tiene que estar en el prompt.

2 Tipos de prompting

CONTENIDO DEL PLAN TIPOS DE PROMPTING: ZERO-SHOT, FEW-SHOT E INSTRUCTIONAL PROMPTING.

Zero-shot es pedir la tarea solo con instrucciones, sin ejemplos. Funciona bien cuando la tarea es común y el formato esperado es fácil de describir. Tiene la ventaja de ser corto y barato en tokens; su limitación es que el modelo interpreta a su manera lo que no se dijo, por ejemplo el tono o la persona gramatical.

Few-shot agrega a las instrucciones unos pocos ejemplos resueltos de entrada y salida. Los ejemplos muestran lo que cuesta describir: el largo, el estilo, cómo tratar un caso especial. En la API de chat se escriben como turnos previos, un mensaje `user` con la entrada y un mensaje `assistant` con la salida esperada. Su costo es que cada ejemplo suma tokens en cada llamada, y que el modelo tiende a imitarlos, incluso en lo que no querías.

Instructional prompting, o prompting instruccional, descompone la tarea en pasos explícitos que el modelo debe seguir en orden. Es útil cuando la tarea tiene varias partes, como leer el ticket, identificar el producto, identificar la falla y recién después redactar.

Estrategia	Qué incluye	Ventaja	Limitación
Zero-shot	Solo instrucciones	Corto, barato y fácil de mantener	El formato y el estilo varían más
Few-shot	Instrucciones y 2 a 5 ejemplos	Fija el formato y los casos difíciles	Más tokens; los ejemplos pueden sesgar

Estrategia	Qué incluye	Ventaja	Limitación
Instruccional	Pasos numerados	Ordena tareas de varias partes	Prompts largos; más difíciles de iterar

EJEMPLO · NUBE SUR · Few-shot para tickets que son preguntas

El ticket T4 no reporta una falla: pregunta cómo exportar el listado de clientes a Excel. Con zero-shot, el modelo a veces responde con una instrucción ("Exporta el listado desde..."). Con dos ejemplos previos, uno de ellos una pregunta resumida como "El usuario pregunta cómo agregar un segundo administrador a su cuenta", el resumen de T4 sale en el mismo formato. Los ejemplos deben ser distintos de los tickets que vas a evaluar, para no contaminar la medición.

3 Buenas prácticas

CONTENIDO DEL PLAN PRÁCTICAS RECOMENDADAS: CLARIDAD, DELIMITACIÓN DEL ROL, CONTEXTO, RESTRICCIONES Y EVITAR AMBIGÜEDAD.

Cinco prácticas explican la mayor parte de la diferencia entre un prompt que sirve y uno que no:

- **Claridad.** Una tarea por prompt, con verbos concretos: "resume en una oración" en vez de "revisa esto".
- **Delimitación del rol.** Decir desde qué papel responde el modelo ("eres analista de soporte") orienta el vocabulario y el nivel de detalle.
- **Contexto.** Lo que el modelo no puede adivinar: para quién es la respuesta, para qué se usará y qué significa "bueno" en este caso.
- **Restricciones.** Lo que no puede hacer: inventar datos, proponer soluciones, superar un largo, cambiar de idioma.
- **Evitar ambigüedad.** Reemplazar palabras como "breve" o "detallado" por medidas ("máximo 25 palabras") y separar las instrucciones de los datos con marcas claras, como "Ticket:".

EJEMPLO · NUBE SUR · El prompt corregido de la actividad 1

```
[system] Eres analista de soporte de Nube Sur. Resume el ticket en UNA
oración de máximo 25 palabras, en tercera persona. Indica qué falla, en
qué producto y en qué condición ocurre. No propongas soluciones ni
agregues datos que no estén en el ticket.
[user] Ticket: <ticket limpio>
```

Tiene rol ("analista de soporte"), tarea y medida ("una oración de máximo 25 palabras"), contexto de lo que importa ("qué falla, en qué producto y en qué condición") y dos restricciones explícitas.

ERROR FRECUENTE · Corregir con la respuesta en vez de con la regla

Ante un resumen que inventó "reinstale la aplicación", es tentador agregar al prompt la solución correcta. Eso arregla un ticket y rompe los demás. La corrección general es la restricción: "No propongas soluciones ni agregues datos que no estén en el ticket".

4 Formato de salida

CONTENIDO DEL PLAN FORMATEO DE SALIDAS: LISTAS, JSON Y ESTRUCTURA TABULAR.

Cuando una persona lee la respuesta, el formato es una cuestión de comodidad. Cuando la lee **otro programa**, es una condición para que funcione. Si tu código espera un campo `prioridad` y el modelo responde con un párrafo, el programa falla. Por eso conviene pedir el formato de manera explícita y, si es posible, mostrarlo.

Los formatos más útiles son tres. Las **listas** sirven para enumerar pasos o puntos y son fáciles de leer. **JSON** es el formato para integrar con código: se pide indicando los campos, sus tipos y los valores permitidos, y el programa lo lee con `json.loads`. La **estructura tabular**, como una tabla en Markdown o un CSV, sirve para comparar varios elementos en las mismas columnas.

```
[system] Eres analista de soporte de Nube Sur. Devuelve SOLO un objeto
JSON válido, sin texto antes ni después, con estos campos:
  "resumen": una oración de máximo 25 palabras,
  "producto": el producto afectado, o null si no se menciona,
  "prioridad": "alta", "media" o "baja".
```

EJEMPLO · NUBE SUR · Salida lista para el tablero

Para el ticket T3, sobre la API de pagos con error 504 que afecta a clientes de dos países, la respuesta esperada es `{"resumen": "La API de pagos devuelve error 504 en parte de las llamadas desde las 09:00 en Chile y Perú.", "producto": "API de pagos", "prioridad": "alta"}`. El servidor la lee con `json.loads` y la muestra en el tablero de la mesa de ayuda sin intervención humana.

ERROR FRECUENTE · Confiar en que el JSON siempre llega bien

Aunque lo pidas, el modelo puede agregar una frase antes del JSON o cortar la respuesta si se queda sin tokens. Valida siempre: envuelve `json.loads` en un `try` y define qué hacer si falla. Algunas APIs permiten además exigir JSON con un parámetro (en OpenAI, `response_format`); revisa en la documentación qué modelos lo aceptan.

5 Ejemplos prácticos

CONTENIDO DEL PLAN EJEMPLOS PRÁCTICOS: PROMPT PARA GENERAR CÓDIGO, RESUMEN, VALIDACIÓN Y REFORMULACIÓN.

Las mismas prácticas se aplican a tareas distintas. Estos cuatro prompts son del trabajo diario de un equipo como el de Nube Sur; en todos, el mensaje de sistema fija rol y reglas, y el de usuario trae los datos.

Tarea	Prompt (resumido)	Qué controla
Generar código	"Escribe una función Python <code>normalizar_espacios(texto: str) -> str</code> que reemplace secuencias de espacios por uno solo y quite los de los extremos. Incluye docstring y dos pruebas con pytest. Responde solo con el código."	Firma, comportamiento, documentación y formato de la respuesta
Resumir	"Resume el ticket en una oración de máximo 25 palabras, en tercera persona. No agregues datos."	Largo, persona y fidelidad
Validar	"Te doy un ticket y un resumen. ¿El resumen contiene información que no está en el ticket? Responde solo SI o NO, y si es SI, la frase inventada."	Respuesta cerrada, fácil de procesar
Reformular	"Reescribe esta respuesta para el cliente en tono cordial y formal, sin cambiar los datos técnicos ni los plazos."	Tono, sin alterar el contenido

EJEMPLO · NUBE SUR · Validar con un segundo prompt

El resumen "Hay un problema con la app. Se recomienda reinstalarla" pasa por el prompt de validación junto con su ticket. La respuesta esperada es "SI: se recomienda reinstalarla". Un prompt que valida la salida de otro es una técnica simple para detectar invenciones antes de mostrarlas.

ERROR FRECUENTE · Pedir código sin decir cómo se entrega

Si el prompt de código no dice "responde solo con el código", el modelo suele agregar explicaciones antes y después. Para pegarlo en un archivo tendrás que limpiarlo a mano. Pide el formato de entrega igual que pides el comportamiento.

6 Parámetros

CONTENIDO DEL PLAN PROMPT Y PARÁMETROS: CÓMO INFLUYEN TEMPERATURE, MAX_TOKENS Y TOP_P.

El prompt dice qué hacer; los **parámetros de generación** controlan cómo elige el modelo cada token. Se envían en el mismo cuerpo de la solicitud y su efecto se suma al del prompt.

- **temperature** controla la variedad. Con valores bajos (0 a 0,3) el modelo elige casi siempre los tokens más probables y las respuestas se parecen mucho entre corridas.

Con valores altos (sobre 0,8) aparecen opciones menos probables: más variedad, pero también más riesgo de inventar. En la API de OpenAI va de 0 a 2.

- **max_tokens** fija el largo máximo de la respuesta, en tokens. No hace que el modelo resuma más: si se alcanza el límite, la respuesta se corta y el campo `finish_reason` de la respuesta dice `length`. Algunos modelos recientes usan el nombre `max_completion_tokens`.
- **top_p** limita la elección a los tokens más probables cuya probabilidad acumulada llega a ese valor. Con 0,1 solo se consideran las opciones de arriba; con 1 se consideran todas. La documentación de OpenAI recomienda ajustar `temperature` o `top_p`, no los dos a la vez.

EJEMPLO · NUBE SUR · Parámetros para el resumidor

Para resumir, que es extraer y no crear, Nube Sur usa `temperature` 0,2: el mismo ticket da casi el mismo resumen en cada corrida. El límite es de 60 tokens, porque una oración de 25 palabras en español ocupa bastante menos. En la actividad 2 vas a comparar 0,2 con 0,9 en los mismos cinco tickets y a medir la diferencia.

ERROR FRECUENTE · Usar max_tokens para acortar

Poner `max_tokens` en 15 no produce resúmenes cortos: produce resúmenes cortados a la mitad. El largo se controla en el prompt ("máximo 25 palabras"); `max_tokens` es un tope de seguridad, con margen. Revisa también en la documentación qué parámetros acepta el modelo que uses: algunos modelos de razonamiento no permiten cambiar `temperature`.

7 Limitaciones

CONTENIDO DEL PLAN LIMITACIONES: ALUCINACIONES, FALTA DE CONTROL Y SENSIBILIDAD AL WORDING.

Un buen prompt reduce los problemas, pero no los elimina. Conocer las limitaciones de los modelos generativos es parte de diseñar con ellos.

Las **alucinaciones** son respuestas que suenan correctas pero incluyen datos falsos o inventados. Ocurren porque el modelo genera texto probable, no texto verificado. En un resumen aparecen como soluciones, causas o cifras que no estaban en el ticket. Se reducen con restricciones explícitas, `temperature` baja y validación posterior, pero no desaparecen.

La **falta de control** significa que no hay garantía de que el modelo cumpla todas las instrucciones siempre. La misma entrada puede dar salidas distintas, un formato pedido puede no respetarse y una regla puede ignorarse en un caso raro. Por eso el código que usa la respuesta debe validarla.

La **sensibilidad al wording**, o a la redacción, es que un cambio pequeño de palabras cambia el resultado. "Resume el ticket" y "Haz un resumen breve del ticket" pueden producir largos y estilos distintos. Por eso un prompt se prueba con varios casos y se congela cuando funciona: cambiarlo "para que quede más bonito" es cambiar el comportamiento.

EJEMPLO · NUBE SUR · Las dos salidas del prompt rechazado

Con Resume esto: <ticket> y temperature 1,0, el ticket de la app móvil que se cierra en Android 14 produjo dos salidas. La primera inventó una solución (alucinación). La segunda, "App falla.", no dijo ni la plataforma ni la condición: el modelo no tuvo control sobre el largo. Un solo prompt mostró dos limitaciones en dos corridas.

8 Iterar

CONTENIDO DEL PLAN ITERACIÓN Y AJUSTE DE PROMPTS SEGÚN RESULTADOS OBTENIDOS.

Un prompt rara vez funciona a la primera. Iterar es mejorarlo con evidencia: probar, observar, cambiar y volver a probar. Tres reglas lo hacen útil. **Cambia una cosa a la vez**, para saber qué produjo el efecto. **Prueba siempre con los mismos casos**, incluidos los difíciles. Y **registra** cada versión: qué cambiaste, por qué y qué efecto tuvo, en términos de claridad, estructura o control del comportamiento.

Versión	Cambio	Efecto observado
v1	Resume esto: <ticket>	Resúmenes de 3 a 5 oraciones, a veces con soluciones inventadas. Falta control
v2	Rol + "una oración, máximo 25 palabras" + "no agregues soluciones"	Largo correcto; T4 sale como instrucción y no como pregunta. Mejora la estructura
v3	"En tercera persona, indica qué falla y en qué producto" + dos ejemplos	T4 queda "El usuario pregunta cómo...". Formato uniforme. Mejora la claridad

EJEMPLO · NUBE SUR · Decidir con números

En la actividad 2, cada versión se mide contra los resúmenes de referencia que escribió el equipo de soporte, con métricas que verás en el aprendizaje esperado 4. Si v3 con temperature 0,2 obtiene el mejor promedio y además no inventa datos en ningún ticket, esa es la configuración que se recomienda, con la tabla de resultados como respaldo.

ERROR FRECUENTE · Cambiar todo a la vez

Si en una misma iteración cambias el rol, agregas ejemplos y bajas la temperature, y el resultado mejora, no sabes cuál de los tres cambios sirvió. La próxima vez que algo falle no sabrás qué tocar. Una variable por versión.

En síntesis

- Un prompt es la especificación de la tarea: qué entra, qué sale y qué no puede pasar.
- En las APIs de chat, el mensaje de sistema fija rol y reglas, y el de usuario trae la tarea y los datos.
- Zero-shot usa solo instrucciones; few-shot agrega ejemplos resueltos; el prompting instruccional descompone la tarea en pasos.
- Claridad, rol, contexto, restricciones y medidas concretas evitan la mayoría de los problemas.
- Si otro programa lee la respuesta, pide JSON con campos definidos y válidalo siempre.
- temperature controla la variedad, max_tokens el tope de largo y top_p el rango de opciones.
- Alucinaciones, falta de control y sensibilidad a la redacción son límites del modelo: se reducen con diseño y validación, no se eliminan.
- Itera cambiando una cosa a la vez, con los mismos casos y un registro de versiones.

SIGUIENTE PASO

Para practicar

- **Actividad 1, "Un cliente de API para el resumidor", parte C.** Diagnosticas el prompt que no sirve, lo reescribes como zero-shot y justificas los parámetros.
- **Actividad 2, "Laboratorio de prompts y métricas", pasos 2 y 3.** Diseñas un prompt zero-shot y uno few-shot, y registras al menos tres iteraciones con temperature 0,2 y 0,9.
- **Notebook guiado "Laboratorio de prompts", secciones 1 a 9.** Envías el prompt que no sirve, lo reescribes como zero-shot, few-shot e instruccional, pides JSON, mides temperature y max_tokens, y registras tus versiones con un puntaje.
- **Video interactivo "Del prompt a la respuesta".** Predice el efecto de un prompt y de temperature antes de verlo.
- **Documentación oficial.** Las guías de prompting de OpenAI (platform.openai.com/docs) y la referencia de cada endpoint, con los parámetros que acepta cada modelo.

Autocomprobación

Responde con tus palabras antes de mirar las respuestas. Si dudas en alguna, vuelve a la sección que se indica.

- 1 ¿Qué diferencia hay entre un prompt zero-shot y uno few-shot, y en qué caso de Nube Sur conviene el segundo?
- 2 Un resumen dice "Se recomienda reinstalar la aplicación", pero el ticket no menciona ninguna solución. ¿Qué limitación es y qué cambios la reducen?
- 3 ¿Qué valor de temperature eliges para resumir tickets y qué pasa si pones max_tokens demasiado bajo?

RESPUESTAS

1. Zero-shot solo da instrucciones; few-shot agrega ejemplos resueltos de entrada y salida. Conviene few-shot cuando el formato es difícil de describir o hay casos especiales, como los tickets que son preguntas (T4): los ejemplos muestran cómo resumirlos (sección 2).
2. Es una alucinación. Se reduce con una restricción explícita ("no propongamos soluciones ni agregues datos que no estén en el ticket"), con temperature baja y con un prompt de validación que revise la salida (secciones 3, 5 y 7).
3. Un valor bajo, como 0,2, porque resumir es extraer y se busca la misma respuesta en cada corrida. Si max_tokens es muy bajo, la respuesta se corta a la mitad y `finish_reason` indica `length`: el largo se controla en el prompt, no con max_tokens (sección 6).

Glosario

Alucinación	Respuesta que suena correcta pero contiene información falsa o que no estaba en la entrada.
Few-shot	Estrategia de prompting que agrega a las instrucciones algunos ejemplos resueltos de entrada y salida.
Mensaje de sistema	Mensaje con rol <code>system</code> que fija el comportamiento general del modelo: rol, reglas y formato.
Prompt	Entrada que recibe un modelo generativo; contiene la instrucción de la tarea y los datos sobre los que debe trabajar.
Temperature	Parámetro que controla la variedad de la respuesta; valores bajos dan salidas más estables.
Token	Unidad de texto que procesa el modelo; puede ser una palabra, parte de una palabra o un signo.
Top_p	Parámetro que limita la elección a los tokens más probables hasta una probabilidad acumulada.
Zero-shot	Estrategia de prompting que pide la tarea solo con instrucciones, sin ejemplos.